

When you are done, you can tap the **WIFI icon** again and terminate your connection.

Use the Controls

Simply tapping the 8 control buttons will send the programmed commands to the ESP32.

Any messages

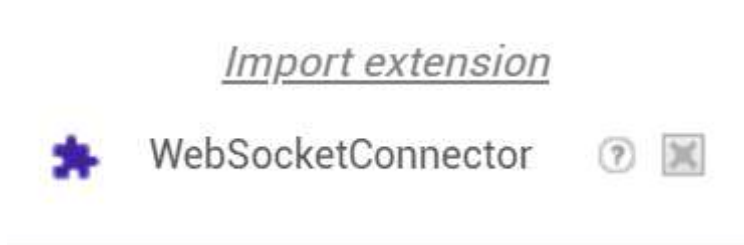
WebSocket Use In APP Inventor

Here, we will examine the code of the APP Inventor APP and look at the WebSocket and project command implementation details. It is assumed that the user is familiar with the APP Inventor programming,

APP Code blocks:

The image displays several blocks of code from the APP Inventor visual programming interface, organized into two columns. The left column contains three main event-driven blocks: 1) 'when Screen1.Initialize' which initializes 'global WiFiClickActive' to false and sets the 'Messages' text to 'Enter remote IP and click WIFI icon.' 2) 'when wifi.Click' which checks if 'global WiFiClickActive' is true; if so, it disconnects the 'WebSocketConnector1'; otherwise, it connects it using a URL constructed from 'ws://', the 'RemoteIP' text, and port '81'. 3) 'when any Button.Click' which sends a message to 'WebSocketConnector1' with the button's text and updates the 'Messages' text to show 'Sent: ' followed by the button text. The right column contains three state-driven blocks for 'WebSocketConnector1': 1) '.Connected' which sets the 'wifi' image to 'WIFlonblk_50p.jpg', hides the 'RemoteIP' label, sets the message to 'Connected...', and sets 'global WiFiClickActive' to false. 2) '.Disconnected' which sets the 'wifi' image to 'WIFloff_50p.jpg', shows the 'RemoteIP' label, sets the message to 'Disconnected...', and sets 'global WiFiClickActive' to false. 3) '.ErrorOccurred' which sets the 'wifi' image to 'WIFloff_50p.jpg', shows the 'RemoteIP' label, sets 'global WiFiClickActive' to false, and sets the message to 'WS error: ' followed by the error details. A fourth block, '.MessageReceived', sets the message to 'Socket message received: ' followed by the received message and sets 'global WiFiClickActive' to false.

WebSocket in the APP is made possible by use of an extension: **WebSocketConnector**.

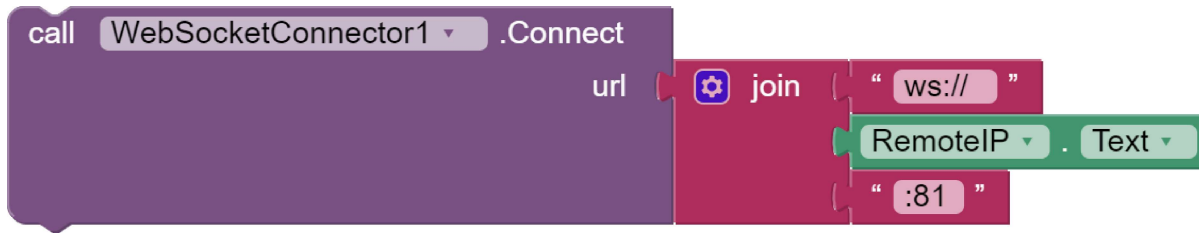


It provides a programming interface that supports the following events:

- Connection
- Disconnection

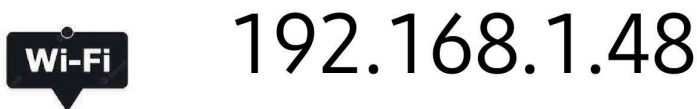
- Message Sending
- Message Reception
- Errors

AI2 WebSocket Connection

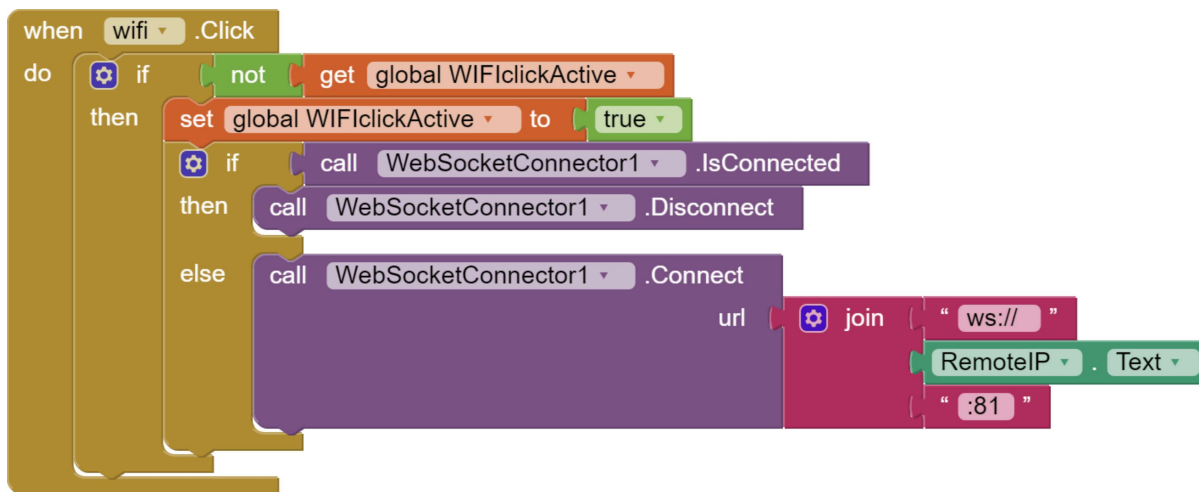


For the Connection, we use the ESP32 server IP Address and the MicroBlocks Websocket port number 81: 192.168.1.48:81. The IP used may differ for your network due to WIFI configuration differences.

The IP address is extracted from the IP Address input field at the top of the APP GUI.



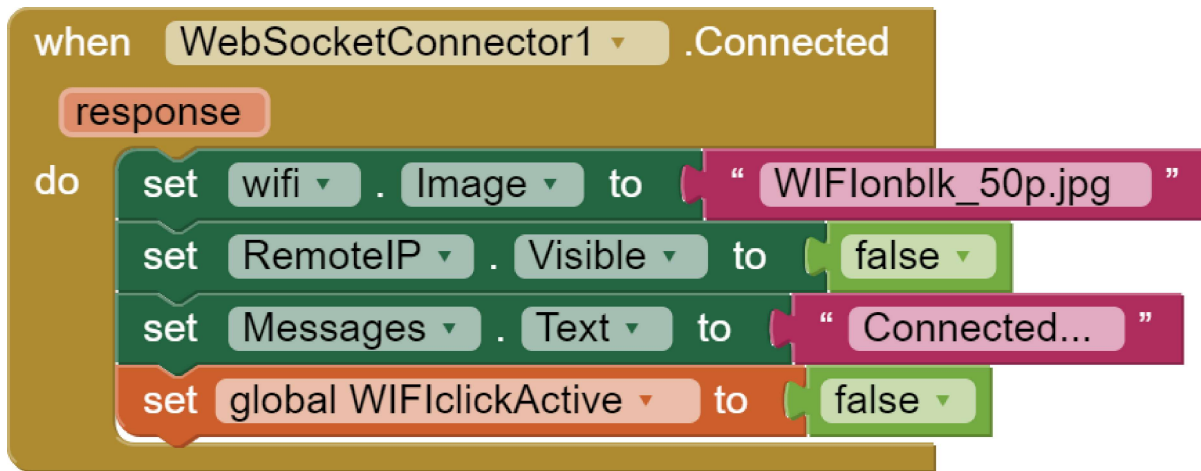
When the **WIFI Icon** is tapped, the APP initiates the connection via the script **when WIFI.click**.



Above script handles the WIFI button click event. It is possible that one inadvertently generates multiple click events by tapping the icon back to back. This would normally cause multiple connections to be initiated; and create a problematic situation.

To prevent this, we implemented a boolean switch **WIFlickActive** that controls the button clicks and their respective event handlers. Initially, this variable is set to false. As soon as a button is clicked, we set this variable to true and do not reset it until all the related click event handling is completed. And the click events only are allowed while the switch is false.

Upon a successful connection establishment, **when WebSocketConnector1 connected** script is executed.



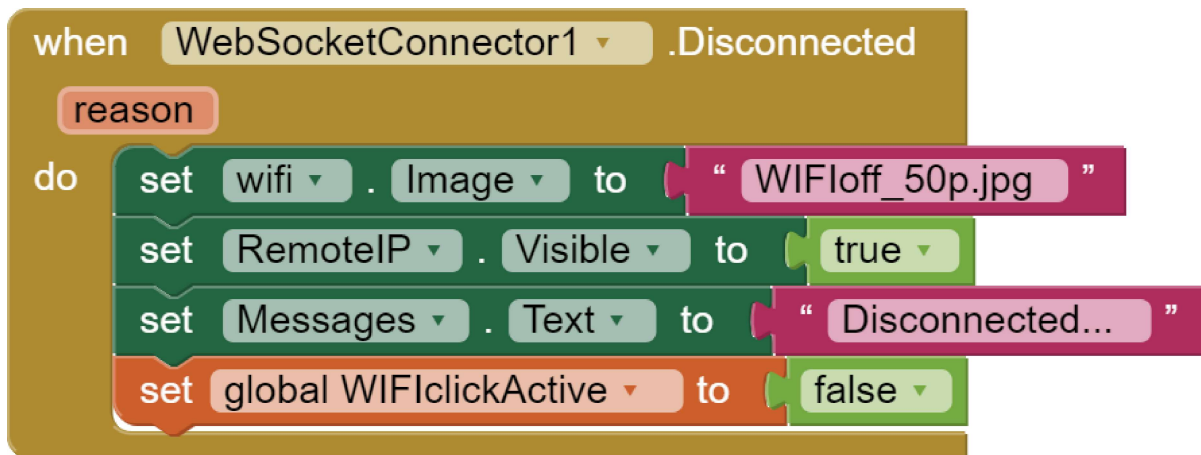
AI2 WebSocket Disconnection



After a connection is established, if the **WIFI icon** is tapped a second time, the link is disconnected. See the code for **when WIFI.click**. script above. The WIFI icon button acts like a toggle switch: if there is no active WebSocket connection, then the connection is done. If the connection is active, then the link is disconnected.



WebSocket link disconnection will cause the script **when WebSocketConnector1 disconnected** to be executed.

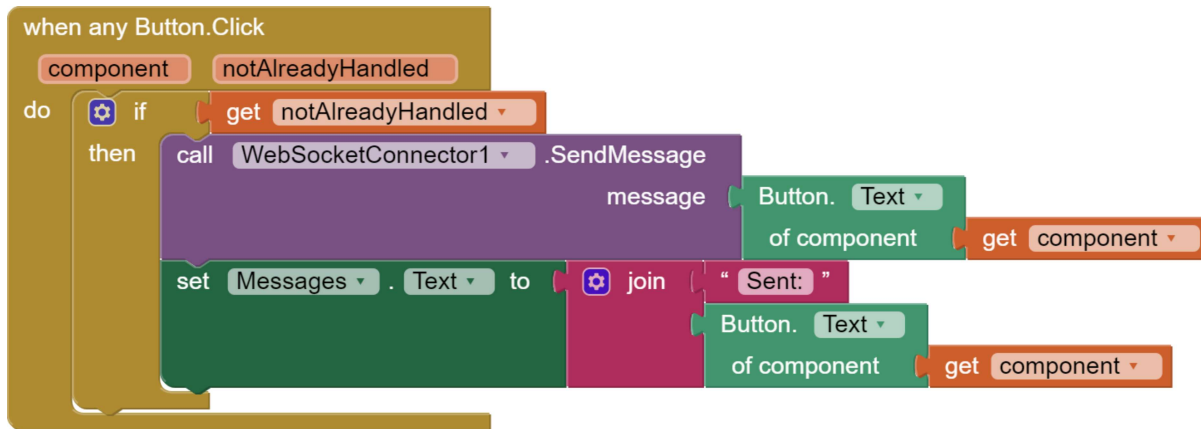


AI2 WebSocket Message Send



Sending a WebSocket message is very simple; it only requires a message text.

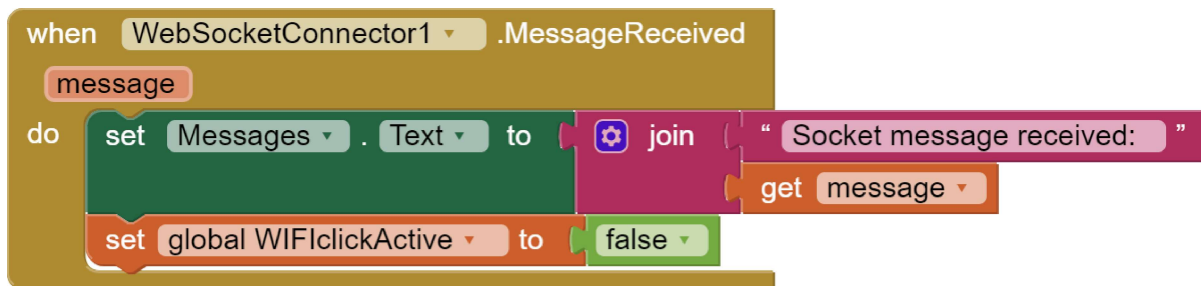
Since we have eight different messages we need to send, we have resorted to a technique to reduce the number of blocks we need to code. **Instead of** using a standard approach of **writing code for each of the eight button controls**, we are using the **generic code technique**.



Since all our commands are activated by button clicks, we use the generic event **when any Button.Click** script. Code written in this script applies to any button clicked. And since all our command buttons do the same thing, it makes our lives easier.

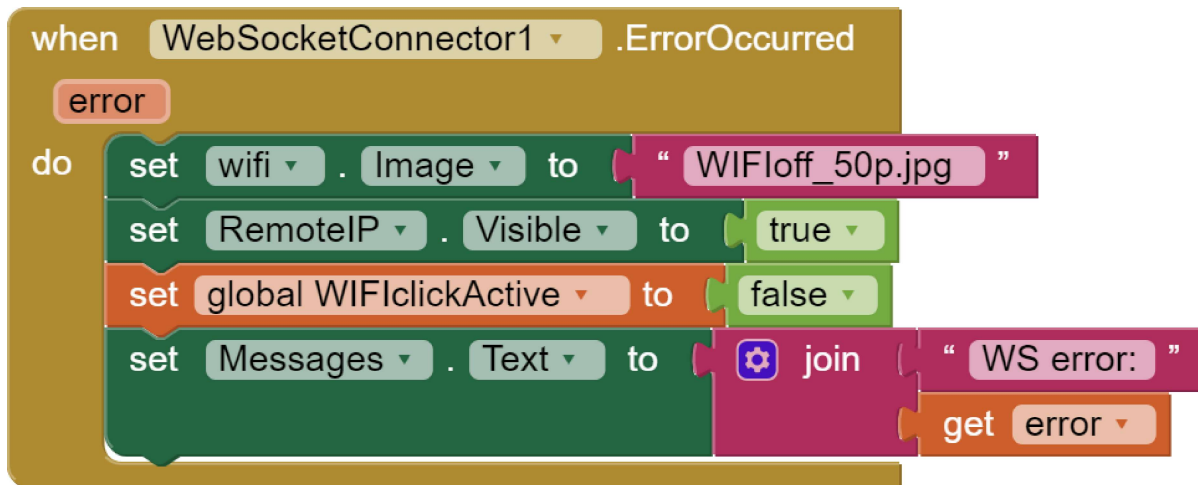
To differentiate the different button actions, we have used another technique of labeling the buttons with their action names: **left, right, up, down, square, triangle, cross, circle**. This gives us the ability to use the button text names as the WebSocket message we need to send. And everything accomplished in a single script.

AI2 WebSocket Message Receive



In this project, we do not receive any messages under normal operation conditions. However, it is possible to receive test message during testing of the project. To accomodate that case, we provide an event handler to display any incoming messages on the message area of the APP, at the bottom of the display.

AI2 WebSocket Errors



In case of any errors during the connection or normal operation of the APP, we provide an event handler to display any error messages on the message area of the APP, at the bottom of the display.

No WIFI, No Problem - Let's fire up our HotSpot !

We have mentioned earlier that there may be cases where the environment you want to exercise control over your devices might not have a WIFI signal available. Specially, if you are in the woods or out in nature or in a far away from civilization location. No worries! You still will be able to experiment with this tutorial by following the directions provided here.

My own WIFI Signal

Let's see how we can activate our own WIFI signal.

To create our own WIFI signal, we will need to select **Button A** after the program starts, or anytime we want to **activate the Hotspot option**.

How Does it Work

To better **visualize the HOTSpot setup**, let's take a look at the picture below.